HANDS-ON LAB REPORT

# AWS Identity and Access Management (IAM)

Tasks 1 to 7: users, policies, roles and least-privilege access

SUBMITTED BY

**Abdul Janeeb K**

DATE

October 2026

# Lab Overview

This report documents seven hands-on exercises in AWS Identity and Access Management (IAM). Each task covers the objective, the steps performed, the evidence captured as screenshots, and what the results show about how AWS evaluates permissions.

## Key concepts used in this lab

- **IAM user:** an identity with long-term credentials (a console password or access keys).
- **Managed policy:** a reusable permission set, either AWS managed (for example ReadOnlyAccess) or customer managed.
- **Least privilege:** granting only the actions a person or workload actually needs.
- **IAM role:** an identity with no permanent credentials; it is assumed and issues temporary credentials.
- **Explicit deny:** always overrides any allow, including a deny from a Service Control Policy (SCP).

## Summary of tasks

| Task | Topic | Learning outcome | Status |
|---|---|---|---|
| **1** | IAM user with console access | IAM users, console authentication, managed policies | **Blocked by SCP** |
| **2** | CLI-only IAM user | Access keys, AWS CLI authentication | **Completed** |
| **3** | Custom IAM policy for S3 | Custom policies, least privilege | **Policy created** |
| **4** | EC2 read-only access | Read-only permissions | **Completed** |
| **5** | Start and stop EC2 only | Action-level permissions | **Evidence pending** |
| **6** | IAM role for EC2 | IAM roles, temporary credentials | **Completed** |
| **7** | Access S3 from EC2 using role | Using IAM roles with EC2 and S3 | **Completed** |

> **Note:** Access keys and a password that were visible in the original screenshots (Tasks 1 and 2) have been redacted in this report.

TASK 1

# IAM User with Console Access

**OBJECTIVE**

Create an IAM user with Management Console access, force a password change at first login, attach the ReadOnlyAccess policy, log in as the new user, and verify that resources can be viewed but not modified.

## Steps performed

- Opened IAM > Users > Create user and entered the user name abduljaneebk.
- Moved through the Set permissions step and reached the Review and create page to check the configuration.
- Confirmed on the IAM users list that the user was created.
- Tried the CLI route for setting a console password: aws iam create-login-profile with the password-reset-required option.

## Screenshots

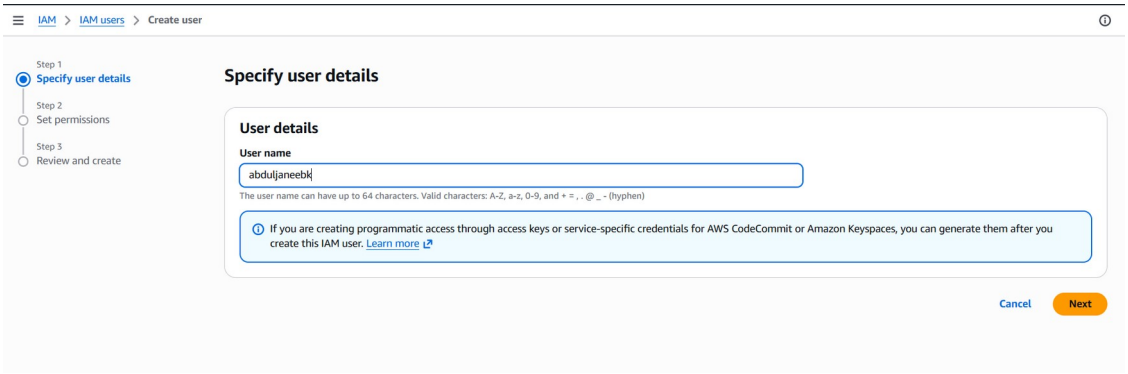



*Figure 1.1: Specify user details with the user name abduljaneebk*

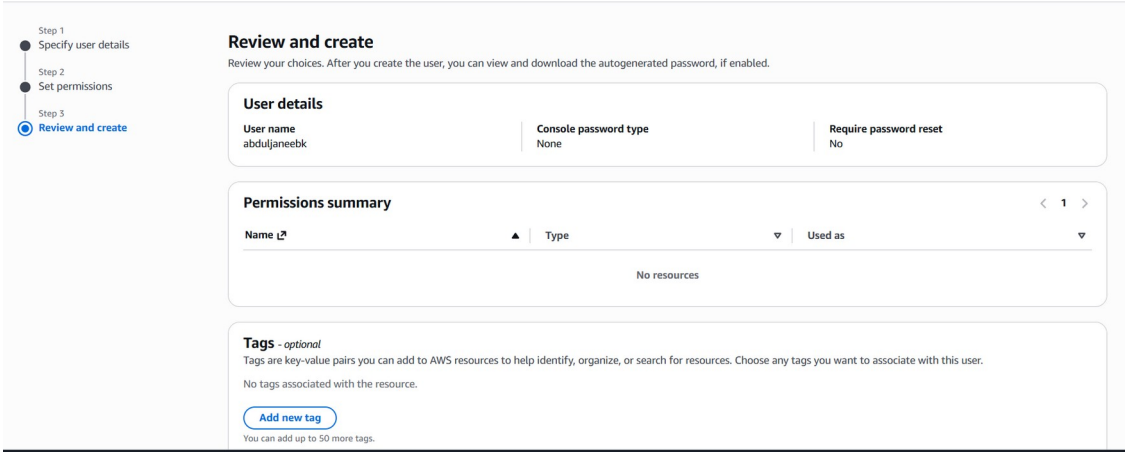



*Figure 1.2: Review and create page for the new user*

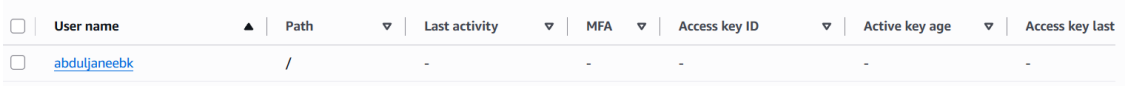



*Figure 1.3: User abduljaneebk listed in IAM users*

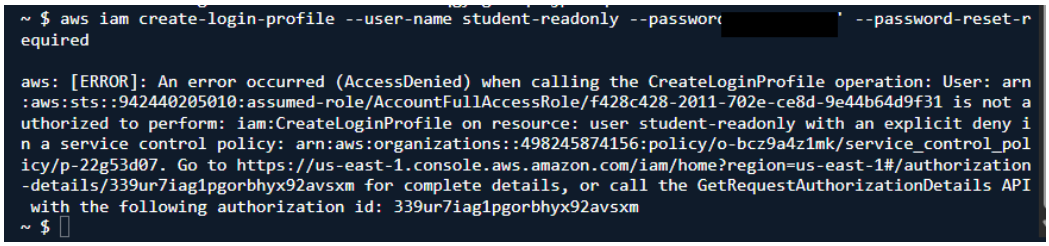



*Figure 1.4: CLI create-login-profile denied (password redacted)*

## Observations

- **Console password:** the console password setup showed "Feature unavailable" in this lab environment, so the user was created with Console password type "None".
- **CLI attempt:** the create-login-profile command failed with AccessDenied.
- **Cause:** the error message names an explicit deny in a service control policy (SCP) applied through AWS Organizations.
- **Why it matters:** an explicit deny in an SCP overrides every allow, even for the AccountFullAccessRole that issued the command.
- **Impact:** console login and the view-but-not-modify test with ReadOnlyAccess could not be completed in this environment.

**Result:** User created; console password setup was blocked by an SCP explicit deny.

**Learning outcome:** *IAM users*, *console authentication*, *managed policies*.

TASK 2

# IAM User without Console Access (AWS CLI)

**OBJECTIVE**

Create a CLI-only IAM user, generate an access key, configure the AWS CLI with the credentials, verify the identity with STS, and list EC2 instances or S3 buckets.

## Steps performed

- Created the IAM user student-cli with no console password (programmatic access only).
- Attached the AWS managed policy AmazonS3ReadOnlyAccess on the permissions step.
- Generated an access key for the user from the Security credentials tab.
- Configured a named profile with `aws configure --profile student-cli` and entered the access key, secret key, region and output format.
- Listed buckets with `aws s3 ls --profile student-cli`.

## Screenshots

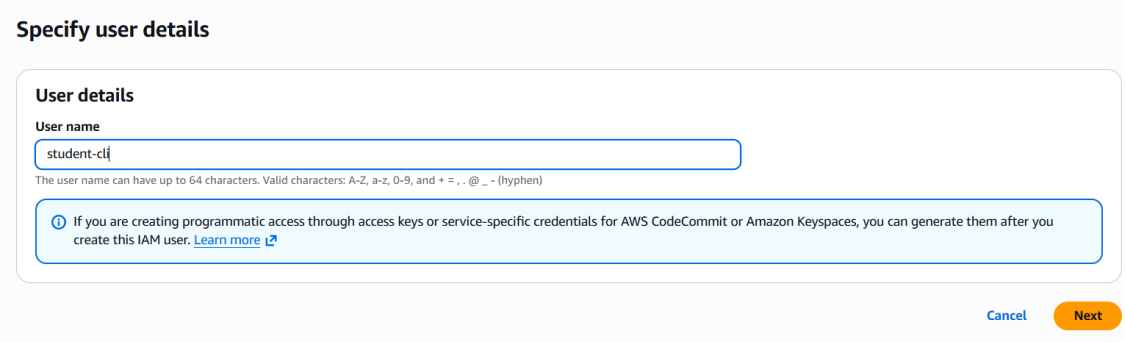



*Figure 2.1: Creating the user student-cli*

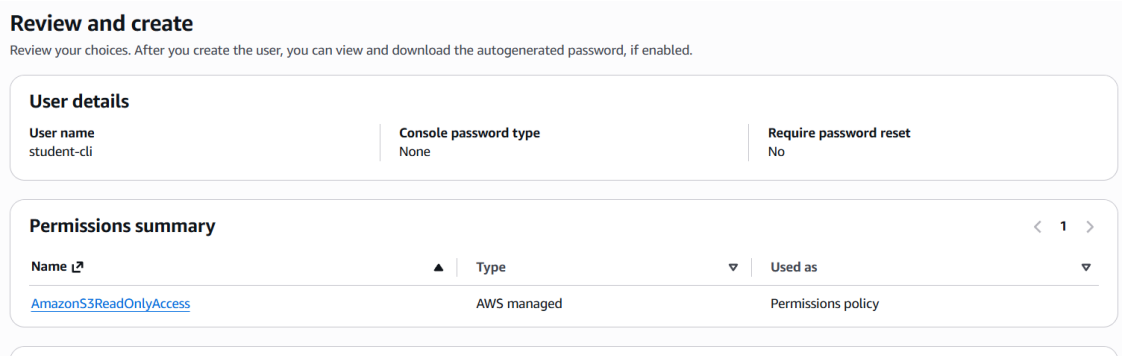



*Figure 2.2: Review page: no console password, AmazonS3ReadOnlyAccess attached*

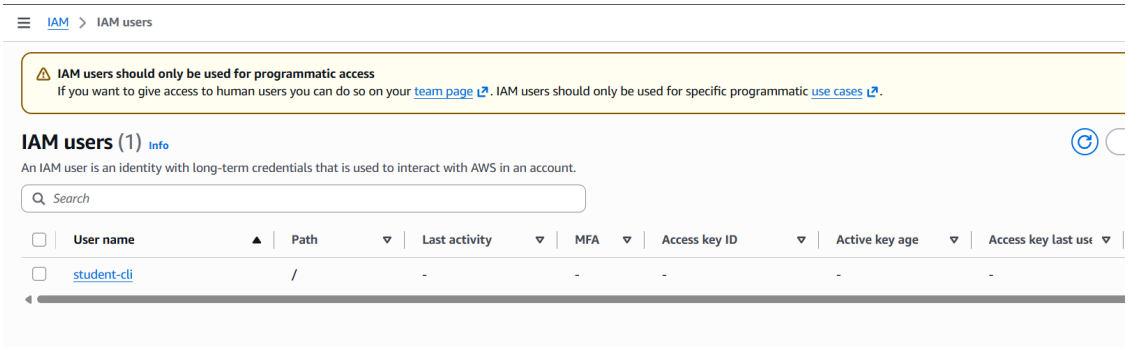



*Figure 2.3: student-cli in the IAM users list*

```
C:\Users\hp>aws configure --profile student-cli
AWS Access Key ID [****************2F6F]:
AWS Secret Access Key [****************eld6]:
Default region name [ap-south-1]: ap-southeast-2
Default output format [json]: json

Configure AWS skills and the AWS MCP server for your AI coding agent(s)? [y/n/never]: n

C:\Users\hp>aws s3 ls --profile student-cli
2026-10-26 13:49:16 abdul-janeeb
2026-10-27 15:57:07 janeeb-969

C:\Users\hp>
```

*Figure 2.4: aws configure and aws s3 ls (keys redacted)*

## Observations

- **No console access:** Console password type is "None", so this user can only authenticate through the CLI or SDKs.
- **Successful operation:** aws s3 ls returned two buckets, abdul-janeeb and janeeb-969, which is allowed by AmazonS3ReadOnlyAccess.
- **Security warning:** the IAM console reminds users that IAM users should only be used for programmatic access.
- **Credential safety:** the secret key is shown only once at creation, so it must be stored securely and never shared.
- **STS check:** identity verification with aws sts get-caller-identity is demonstrated in Task 4.

**Result:** CLI user created and configured; S3 buckets listed successfully.

**Learning outcome:** *Access keys, AWS CLI authentication.*

TASK 3

# Create a Custom IAM Policy

> OBJECTIVE
> Create a JSON policy that allows listing and reading S3 buckets but prevents bucket deletion. Attach it to a user and test both allowed and denied actions.

## Steps performed

- Opened IAM > Policies > Create policy and switched to the JSON editor.
- Wrote a policy with an Allow statement for list and read actions and an explicit Deny for s3:DeleteBucket.
- Saved the policy with the name S3ReadNoDeletePolicy (type: Customer managed).
- Tested read access with `aws s3 ls` and `aws s3 ls s3://janeeb-969` using the student-cli profile in region ap-southeast-2.

## Policy design

The structure below shows how such a policy is written. Check it against the JSON saved in your account before submitting.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowListAndRead",
      "Effect": "Allow",
      "Action": [
        "s3:ListAllMyBuckets",
        "s3:ListBucket",
        "s3:GetBucketLocation",
        "s3:GetObject"
      ],
      "Resource": "*"
    },
    {
      "Sid": "DenyBucketDeletion",
      "Effect": "Deny",
      "Action": "s3:DeleteBucket",
      "Resource": "*"
    }
  ]
}
```

## Screenshots

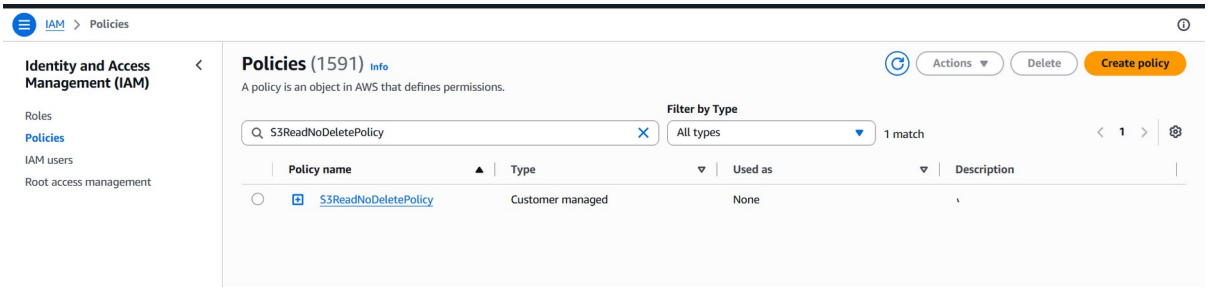



*Figure 3.1: S3ReadNoDeletePolicy shown as a customer managed policy*

```
Microsoft Windows [Version 10.0.22000.2538]
(c) Microsoft Corporation. All rights reserved.

C:\Users\hp>aws s3 ls --profile student-cli --region ap-southeast-2
2026-10-26 13:49:16 abdul-janeeb
2026-10-27 15:57:07 janeeb-969

C:\Users\hp>aws s3 ls s3://janeeb-969 --profile student-cli --region ap-southeast-2
```

*Figure 3.2: Listing buckets and bucket contents with the student-cli profile*

## Observations

- **Customer managed policy:** unlike AWS managed policies, it is created and maintained by you, so it can be tailored exactly.
- **Allowed actions:** listing buckets and reading objects succeeded.
- **Deny wins:** an explicit Deny on s3:DeleteBucket blocks deletion even if another policy grants broader S3 permissions.
- **Least privilege:** only the actions required for browsing and reading are granted.

> **To add:** a screenshot of an attempted bucket deletion (for example aws s3 rb s3://bucket-name) returning AccessDenied would complete the denied-operation evidence for this task.

**Result:** Custom policy created; list and read operations worked as expected.

**Learning outcome:** *Custom policies*, *least privilege*.

TASK 4

# EC2 Read-Only Access

> **OBJECTIVE**
> Provide only EC2 read permissions to a user. Confirm the user can view instances, volumes and security groups but cannot start, stop or terminate instances.

## Steps performed

- Created the IAM user ec2-readonly-user and attached the AWS managed policy AmazonEC2ReadOnlyAccess.
- Verified the identity with `aws sts get-caller-identity --profile ec2-readonly-user`.
- Ran the allowed read commands: `describe-instances`, `describe-volumes` and `describe-security-groups` in region ap-southeast-2.
- Ran the write commands `stop-instances`, `start-instances` and `terminate-instances` with `--dry-run` to confirm they are denied.

## Screenshots: user and permissions

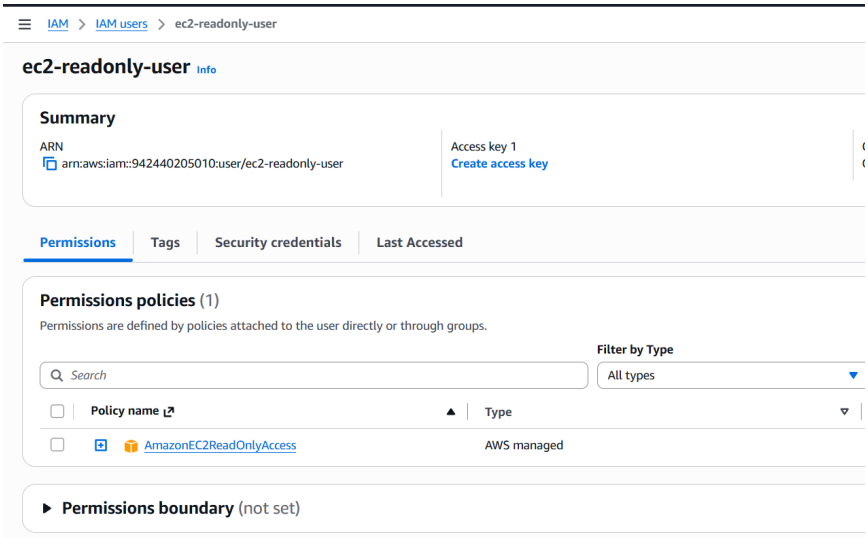



*Figure 4.1: ec2-readonly-user with AmazonEC2ReadOnlyAccess attached*

## Screenshots: allowed operations




*Figure 4.2: sts get-caller-identity and describe-instances*

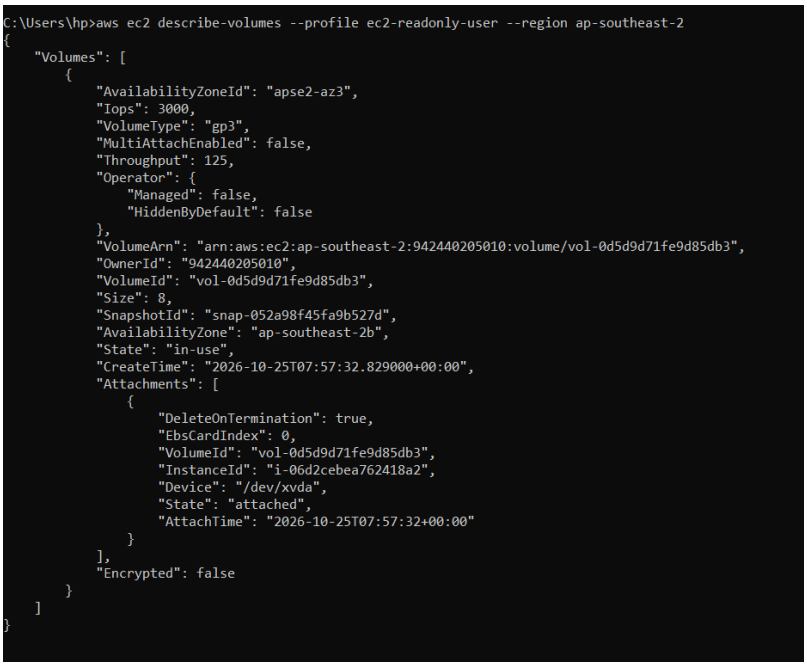



*Figure 4.3: describe-volumes returns the 8 GiB gp3 volume*




*Figure 4.4: describe-security-groups returns the VPC security groups*

## Screenshots: denied operations

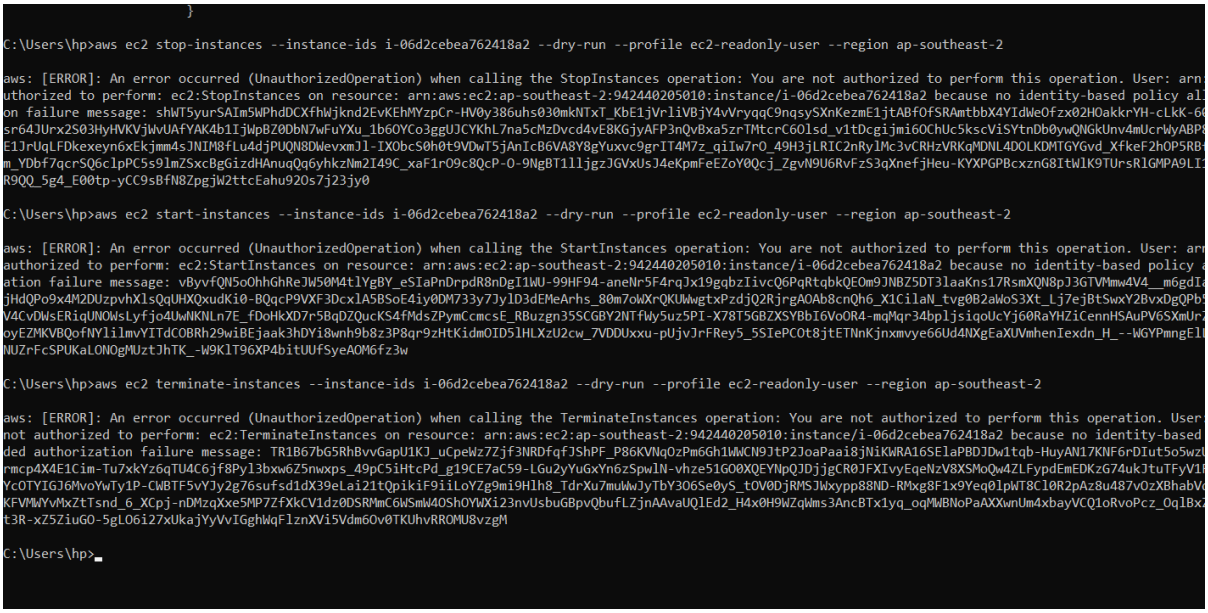



*Figure 4.5: stop, start and terminate all fail with UnauthorizedOperation*

## Observations

- **Identity:** STS returned the ARN of ec2-readonly-user, confirming the profile uses the correct credentials.
- **Allowed:** instances, the EBS volume (vol-0d5d9d71fe9d85db3, 8 GiB, gp3, attached) and security groups were all readable.
- **Denied:** StopInstances, StartInstances and TerminateInstances each returned UnauthorizedOperation because no identity-based policy allows them.
- **Dry run:** the --dry-run flag checks permission without changing the instance, so the test is safe.
- **Pattern:** Describe* actions are read-only, while Start, Stop and Terminate change state and need separate permissions.

**Result:** Read access confirmed; all state-changing actions were denied.
**Learning outcome:** *Read-only permissions.*

TASK 5

# Start and Stop EC2 Only

> OBJECTIVE
> Create a custom policy allowing DescribeInstances, StartInstances and StopInstances. Verify that launching or terminating instances is denied.

## Steps to perform

- Open IAM > Policies > Create policy and use the JSON editor.
- Allow only ec2:DescribeInstances, ec2:StartInstances and ec2:StopInstances.
- Name the policy (for example EC2StartStopOnly) and attach it to a test user.
- Test allowed actions: `describe-instances`, then `stop-instances` and `start-instances` on the instance.
- Test denied actions: `run-instances` and `terminate-instances` should return UnauthorizedOperation.

## Policy design

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "EC2StartStopOnly",
      "Effect": "Allow",
      "Action": [
        "ec2:DescribeInstances",
        "ec2:StartInstances",
        "ec2:StopInstances"
      ],
      "Resource": "*"
    }
  ]
}
```

## Observations

- **Action-level control:** permissions are granted per API action, not per service, so a user can manage the power state without being able to create or delete instances.
- **Implicit deny:** run-instances and terminate-instances are not listed, so they are denied by default.
- **Reference:** the denied-operation output in Figure 4.5 shows the error format to expect.

> **To add:** screenshots of the policy, the successful start and stop commands, and the denied launch and terminate commands were not present in the original file. Add them under this task before submission.

**Learning outcome:** *Action-level permissions.*

TASK 6

# IAM Role for EC2

**OBJECTIVE**

Create an IAM role with S3 read-only permissions, attach it to an EC2 instance, and access AWS services from the instance without configuring access keys.

## Steps performed

- Created the IAM role EC2-S3-ReadOnly-Role with EC2 as the trusted service and S3 read-only permissions.
- Attached the role to the EC2 instance i-06d2cebea762418a2 as its instance profile.
- Connected to the Amazon Linux 2023 instance over SSH. No access keys were configured on the instance.
- Ran `aws sts get-caller-identity` to see which identity the instance uses.
- Ran `aws s3 ls` to read from S3, then tried `aws s3 cp test.txt s3://janeeb-969/test.txt` to write.

## Screenshot

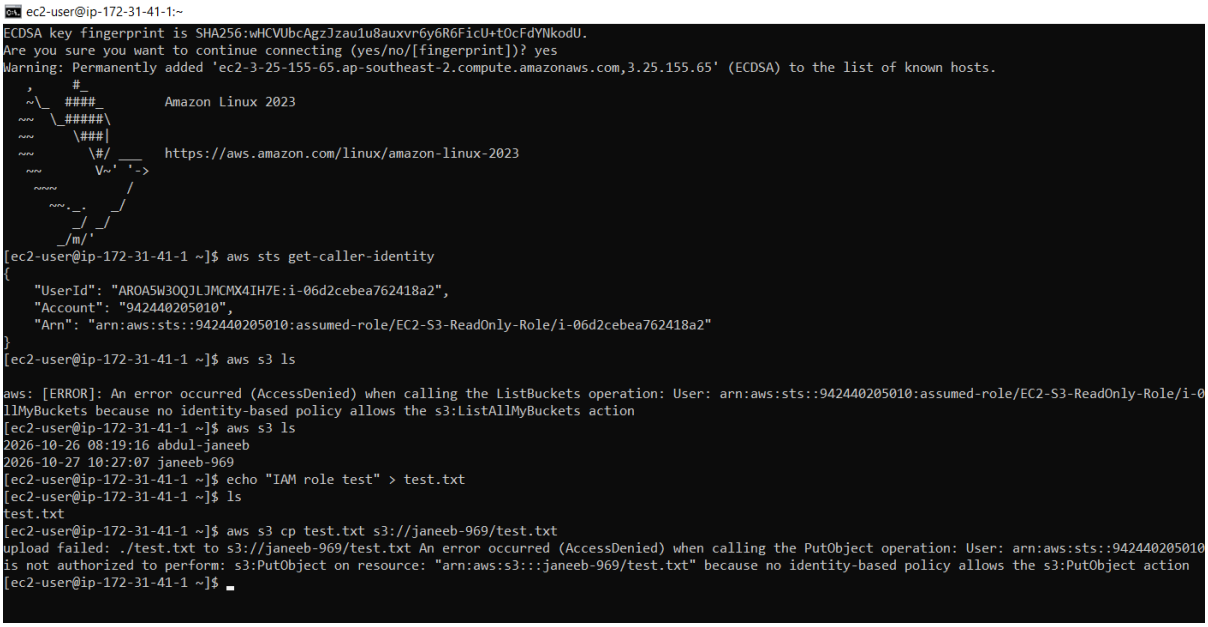



*Figure 6.1: SSH session on the EC2 instance using the role (no access keys)*

## Observations

- **Role in use:** STS returned an assumed-role ARN for EC2-S3-ReadOnly-Role, not an IAM user, which proves the instance is using the role.
- **Temporary credentials:** AWS supplies short-lived credentials to the instance automatically, so nothing secret is stored on disk.
- **First attempt:** the first aws s3 ls returned AccessDenied for ListAllMyBuckets. IAM changes can take a short while to take effect.
- **Second attempt:** the repeated command listed both buckets, abdul-janeeb and janeeb-969.

- **Write denied:** uploading test.txt failed with AccessDenied for s3:PutObject, because the role grants read access only.

**Result:** Instance accessed S3 through the role without keys; read worked and write was denied.
**Learning outcome:** *IAM roles*, *temporary credentials*.

TASK 7

# Access S3 from EC2 Using IAM Role

> OBJECTIVE
> Create an S3 bucket, upload sample files, use the EC2 IAM role to list and download objects, then verify that uploads fail because only read access is granted.

## Steps performed

- Used the S3 bucket janeeb-969, which holds sample files under pets/cat and pets/dog.
- On the EC2 instance, created `sample.txt` and tried to upload it with `aws s3 cp sample.txt s3://janeeb-969/`.
- Listed the bucket with `aws s3 ls s3://janeeb-969/ --recursive`.
- Downloaded an object with `aws s3 cp s3://janeeb-969/pets/cat/cat.jpg ./cat.jpg`.

## Screenshot

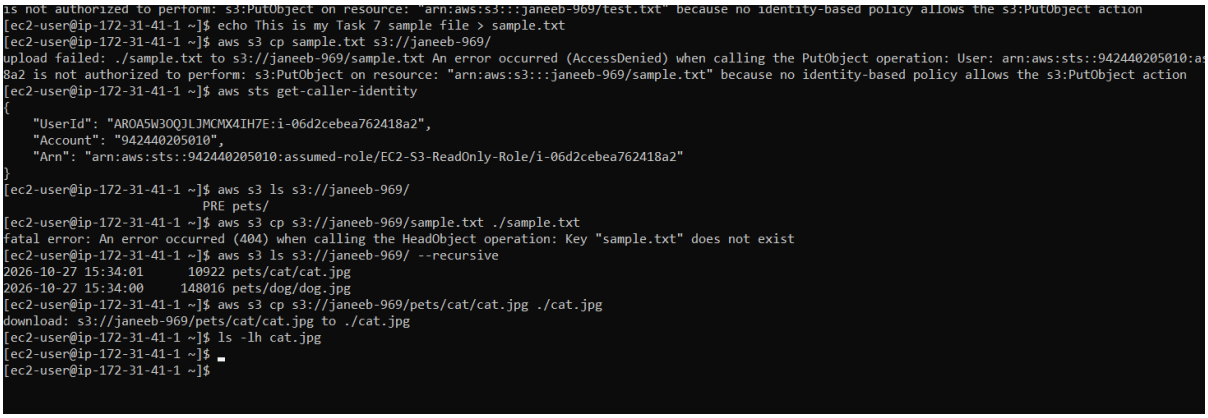



*Figure 7.1: Upload denied, listing and download succeed using the EC2 role*

## Observations

- **Identity:** STS again shows the assumed role EC2-S3-ReadOnly-Role on instance i-06d2cebea762418a2.
- **Upload denied:** sample.txt could not be uploaded because s3:PutObject is not allowed.
- **404 error:** trying to download sample.txt returned "Key does not exist", which is expected because the upload never succeeded.
- **List allowed:** the recursive listing showed pets/cat/cat.jpg (10,922 bytes) and pets/dog/dog.jpg (148,016 bytes).
- **Download allowed:** cat.jpg was downloaded to the instance, which needs only s3:GetObject.

**Result:** Read operations (list, download) succeeded; write operations (upload) were denied, as intended.

**Learning outcome:** *Using IAM roles with EC2 and S3.*

# Conclusion

Across the seven tasks, the lab showed how IAM controls who can do what in an AWS account, and how to test that control with allowed and denied operations.

## Key takeaways

- **Policies decide access:** AWS managed policies give quick, broad permissions; custom policies give precise control.
- **Explicit deny always wins:** Task 1 showed an SCP blocking an action even for a highly privileged role.
- **Least privilege works:** read-only users could view resources but received UnauthorizedOperation or AccessDenied on every write action.
- **Roles are safer than keys:** the EC2 instance reached S3 with temporary credentials and no stored access keys.
- **Testing matters:** running both allowed and denied commands proves that permissions behave as designed.

## Security best practices applied

- Used read-only policies wherever write access was not required.
- Preferred an IAM role over long-term access keys for the EC2 workload.
- Used --dry-run to test destructive EC2 commands safely.
- Redacted access keys and passwords from screenshots in this report.